Pràctica 2

1. Descripció del Problema

L'objectiu d'aquesta pràctica es desenvolupar un sistema que permeti modelar, consultar i actualitzar informació sobre el **càmping Green**. A la Figura 1 es pot veure un mapa esquemàtic del càmping.

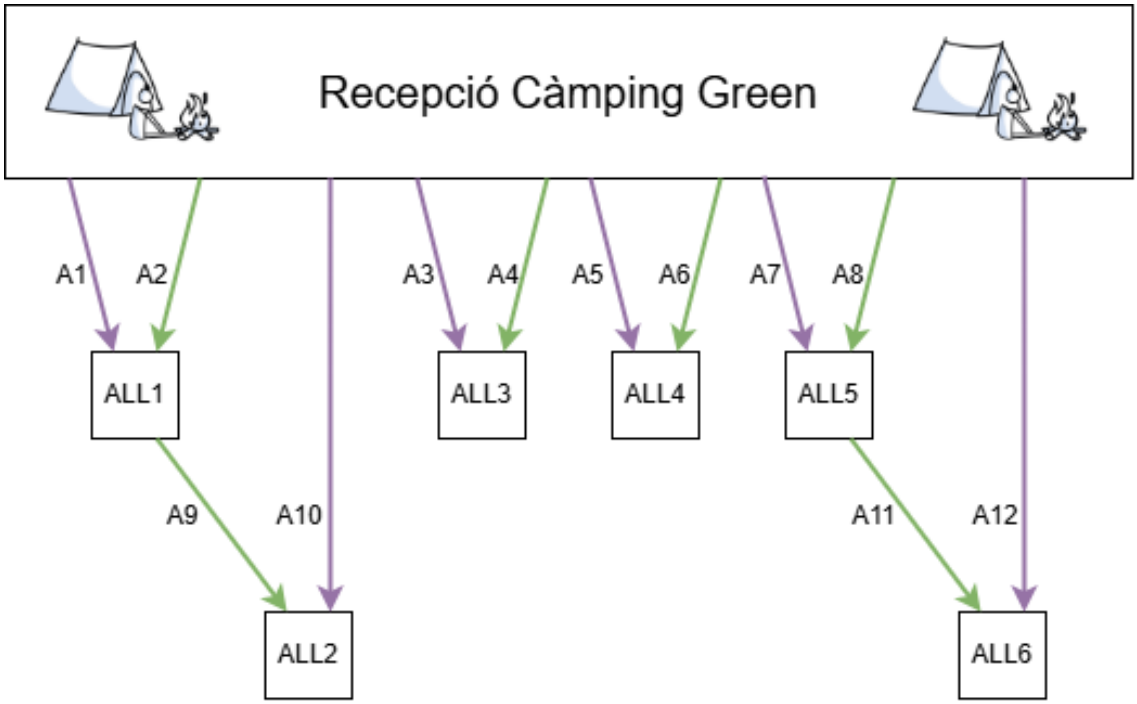

Figura 1. Mapa esquemàtic del càmping *Green*. Els allotjaments estan representats amb un quadrat amb el nom ALL d'allotjament i els accessos estan representats amb línies contínues liles (accés a peu) o verdes (accés amb vehicle) amb els noms A d'accés.

Els allotjaments. El càmping té 6 allotjaments diferents, i tenen la mateixa estructura i propietats que en la pràctica anterior, de manera que es poden tractar de tipus **Parcel·la, Bungalow, Bungalow Premium, Glamping, i Mobile-Home**. A més, disposen de dues propietats extres: l'estat de l'allotjament (que pot ser Operatiu o No operatiu), i un estat de la il·luminació (que pot ser 100%, 50%, o 0%). Els detalls de cada allotjament, així com l'estat actual dels mateixos, es pot trobar a la Taula 1.

Tipus	Nom	Id	Estada mínima (dies) Temp. baixa	Estada mínima (dies) Temp. alta	Propietats	Estat	Il·luminació
Parcel·la	Parcella Nord	ALL1	2	4	Mida 64 m^2 amb punt de connexió elèctrica.	Operatiu	100%
Parcel·la	Parcella Sud	ALL2	2	4	Mida 64 m^2 amb punt de connexió elèctrica.	Operatiu	100%
Bungallow	Bungallow Nord	ALL3	4	7	Mida 22 m^2, 2 habitacions, per 4 persones, 1 plaça de pàrquing, terrassa, televisió, aire fred.	Operatiu	100%
Bungallow Premium	Bungallow Sud	ALL4	4	7	Mida 27 m^2, 2 habitacions, per 6 persones, 1 plaça de pàrquing, terrassa, televisió, aire fred, llençols i tovalloles incloses, codi Wifi.	Operatiu	100%
Glamping	Glamping Nord	ALL5	3	5	Mida 20 m^2, 1 habitacions, per 2 persones, de tela, casa per mascota.	Operatiu	100%
Mobil-Home	Mobil-Home Sud	ALL6	3	5	Mida 22 m^2, 2 habitacions, per 4 persones, terrassa amb barbacoa	Operatiu	100%

Taula 1: Informació detallada sobre els allotjaments del càmping *Green* (estats a l'inici).

Els accessos. El càmping disposa de 12 accessos que permeten als clients desplaçar-se per arribar als allotjaments des de la recepció (tal com es pot veure a la Figura 1). Cada accés té un nom i guarda la informació de si està obert o no i la llista d'allotjaments a les que permet l'accés.

Per tal de facilitar el transport de material als diferents allotjaments, hi ha accessos que permeten l'accessibilitat als allotjaments amb vehicle. En concret, al càmping hi ha 4 tipus d'accessos amb diferent accessibilitat amb vehicle (Acc): carreteres asfaltades (Acc Sí), carreteres de terra (Acc Sí), camins asfaltats (Acc No), i camins de terra (Acc No).

Els accessos es divideixen en accessos asfaltats (carreteres asfaltades i camins asfaltats) que guarden els metres quadrats d'asfalt, i els accessos de terra (carreteres de terra i camins de terra) que guarden la longitud del camí en metres.

A més, cal guardar certa informació sobre les característiques de dos tipus d'accessos: el pes màxim dels vehicles que poden passar per les carreteres asfaltades, i l'amplada de les carreteres de terra. Els detalls dels accessos del càmping es mostren a la Taula 2.

Nom	Tipus	Característiques	Accessibilitat	Estat	Accés a
A1	Camí asfaltat	200 m^2 d'asfalt	No	Obert	ALL1; ALL2
A2	Carretera asfaltada	800 m^2 d'asfalt, pes màxim 10000 kg.	Sí	Obert	ALL1; ALL2
A3	Camí de terra	Longitud 100 metres.	No	Obert	ALL3
A4	Carretera de terra	Longitud 200 metres, amplada 3 metres.	Sí	Obert	ALL3
A5	Camí asfaltat	350 m^2 d'asfalt	No	Obert	ALL4
A6	Carretera asfaltada	800 m^2 d'asfalt, pes màxim 12000 kg.	Sí	Obert	ALL4
A7	Camí asfaltat	100 m^2 d'asfalt	No	Obert	ALL5; ALL6
A8	Carretera asfaltada	800 m^2 d'asfalt, pes màxim 10000 kg.	Sí	Obert	ALL5; ALL6
A9	Camí de terra	Longitud 50 metres.	No	Obert	ALL2
A10	Carretera de terra	Longitud 400 metres, amplada 4 metres.	Sí	Obert	ALL2
A11	Camí de terra	Longitud 80 metres.	No	Obert	ALL6
A12	Carretera de terra	Longitud 800 metres, amplada 5 metres.	Sí	Obert	ALL6

Taula 2: Informació detallada sobre els accessos existents al càmping Green (estats a l'inici).

Les tasques de manteniment. D'altra banda, es volen gestionar les tasques de manteniment que es realitzen als allotjaments i que causen que aquests no estiguin operatius momentàniament. Hi ha 4 tipus de tasques de manteniment: *reparació* (s'ha de reparar algun aspecte de l'allotjament), *neteja* (l'allotjament requereix una neteja), *revisió tècnica* (s'ha de revisar algun element de l'allotjament), i *desinfecció* (s'ha de desinfectar l'allotjament). Cada tasca de manteniment té un número únic que serveix per identificar-la i a més guarda l'allotjament on s'ha de realitzar, el tipus de tasca, la data, i els dies esperats per realitzar-la.

La gestió automàtica de la il·luminació. Com que es tracta d'un càmping "verd" es vol minimitzar l'ús energètic mitjançant l'optimització dels recursos d'il·luminació. Per això, l'estat de la il·luminació dels allotjaments i els seus accessos canviarà depenent de les tasques de manteniment que es realitzin als allotjaments.

Per tal de gestionar de forma eficaç la il·luminació del càmping, s'ha desenvolupat un sistema automàtic. Aquest sistema entra en acció cada vegada que s'afegeix o es completa una tasca de manteniment en algun dels allotjaments, i controla que els allotjaments i els accessos actualitzin automàticament el seu estat en funció de la nova tasca. S'entén que **un allotjament no pot tenir més d'una tasca de manteniment**. Llavors, a l'hora de crear i afegir una tasca de manteniment es verificarà que no existeixi ja una tasca de manteniment sobre aquest allotjament.

Quan s'afegeix una tasca de manteniment, **l'allotjament no estarà operatiu i el tipus de tasca** (que pot ser: reparació, neteja, revisió tècnica, desinfecció) **determinarà l'estat de la seva il·luminació** (que es deixarà al: 50%, 100%, 50% o 0%, respectivament per cada tipus de tasca).

A més, quan s'afegeix una tasca de manteniment a un allotjament, també **s'han de tancar els accessos que permeten arribar a aquest allotjament**. Per fer-ho s'haurà de consultar la llista d'allotjaments dels accessos (última columna de la Taula 2 anomenada *Accés a*). S'ha de tenir en compte que no s'ha de tancar un accés si aquest permet accedir a un altre allotjament a més de l'allotjament on s'ha afegit la tasca. Per exemple, si s'afegeix una tasca de manteniment a l'allotjament ALL1, l'accés A2 no s'ha de tancar, perquè també permet l'accés a l'allotjament ALL2. En canvi, una tasca a l'allotjament ALL3 tancaria els accessos A3 i A4.

S'entén que quan un accés està tancat la seva il·luminació està apagada (i per tant s'aconsegueix l'estalvi desitjat) i quan està obert la seva il·luminació està oberta.

A l'inici, al càmping no hi ha cap tasca de manteniment, tots els allotjaments estan operatius i il·luminats, i tots els accessos estan oberts i il·luminats com es pot veure a la Taula 1 i Taula 2.

2. Objectiu i Funcionalitats

L'objectiu d'aquesta pràctica és modelar la informació corresponent al càmping *Green* utilitzant el paradigma de programació orientat a objectes. L'aplicació desenvolupada haurà d'oferir el següent conjunt de funcionalitats:

1. *Llistar la informació de tots els allotjaments*
2. *Llistar la informació dels allotjaments operatius*
3. *Llistar la informació dels allotjaments no operatius*
4. *Llistar la informació dels accessos oberts*
5. *Llistar la informació dels accessos tancats*
6. *Llistar la informació de les tasques de manteniments actives*
7. *Afegir una tasca de manteniment*
   - L'aplicació haurà de demanar a l'usuari que introdueixi el número de la tasca, el nom de l'allotjament on es vol realitzar la tasca, el tipus de tasca que s'afegeix, la data en que s'afegeix, i els dies esperats per realitzar la tasca. A més, aquesta operació pot produir errors que s'hauran de gestionar.
8. *Completar una tasca de manteniment*
   - L'aplicació haurà de llistar les tasques de manteniment que hi ha actives a la llista de tasques i demanar a l'usuari que doni el número de la tasca que s'ha completat.
9. *Calcular i mostrar el número total d'accessos que **NO** proporcionen accessibilitat amb vehicle*
10. *Calcular i mostrar el número total de metres dels accessos de terra.*
    - És a dir, la suma de metres (longitud) que tenen les carreteres de terra i els camins de terra.
11. *Guardar càmping*
    - Guarda les dades del càmping en un fitxer.
12. *Recuperar càmping*

- Carrega d'un fitxer les dades del càmping prèviament guardades.

13. *Sortir de l'aplicació*

Aquesta pràctica te similituts amb la pràctica 1 i per tant es podran reutilitzar classes de la pràctica 1.

Com vau fer a la pràctica 1, organitzareu el codi en dos paquets [paquet Vista] (**prog2.vista**) o [paquet Model] (**prog2.model**). Les classes de la vista s'encarregaran de la interacció amb l'usuari i les classes del model contindran totes les dades que s'han de manipular i faran totes les accions que afectin a les dades. Des de la classe principal de la vista es demanaran per pantalla els valors dels atributs necessaris per passar-los a les classes del model. Tingueu en compte que **tots els prints s'hauran de fer a la vista, mitjançant el tractament d'excepcions**, i no hi haurà cap print a cap classe del model.

3. Material pel lliurament

Per aquesta pràctica us proporcionem un codi base que conté les classes i interfícies necessàries per la pràctica:

- ***IniciadorCamping***
- ***Menu***
- ***ExempleMenu (Per veure com s'ha d'utilitzar la classe Menu)***
- ***InAcces***
- ***InAllotjament***
- ***InCamping***
- ***InLListaAllotjaments***
- ***InLlistaAcessos***
- ***InLlistaTasquesManteniment***
- ***InTascaManteniment***

Trobareu aquest codi al Campus Virtual junt amb aquest enunciat en la secció de PRÀCTIQUES: "Pràctica 2 (enunciat i codi base)". L'heu de descarregar i afegir-lo al vostre projecte.

Tal com es veurà en el punt 5 de descripció del projecte, hi haurà algunes classes de la pràctica 1 que s'hauran de reutilitzar i ampliar (***Allotjament*** i les seves classes filles, ***Camping***) i algunes que no (***Reserva***, ***LlistaReserves***, ***GestorCamping***). A part, hi haurà una serie de classes noves per gestionar els accessos i les tasques de manteniment dels allotjaments.

4. Descripció del Codi Base

Al Campus Virtual de l'assignatura podràs trobar les classes i interfícies que et serviran de base. Un cop hagis creat el teu projecte IntelliJ (veure secció **4.1**) has d'afegir aquestes classes i interfícies per poder-les fer servir.

4.1 Creació del Projecte Base amb IntelliJ

El primer pas serà crear un projecte al IntelliJ, al qual li heu de posar com a nom Cognom1Nom1Cognom2Nom2, on 1 i 2 fa referència als dos membres de la parella, i tenint en compte les següents consideracions:

- La primera lletra de cada part en majúscula i la resta en minúscula.
- Eviteu utilitzar accents i caràcters del tipus ñ o ç.

  Per exemple, una estudiant amb nom Dolça Martínez Castaña, hauria de crear un projecte amb el nom MartinezCastanaDolca.
- La classe principal s'ha de dir ***IniciadorCamping***, i ha d'estar en el paquet **prog2.vista**.

4.2 Codi Base: Classe *IniciadorCamping*

Aquesta classe té com a responsabilitat llançar el bucle principal de l'aplicació. La classe de la vista ***IniciadorCamping*** té un mètode estàtic ***main()*** on es crea un objecte de tipus ***VistaCamping*** anomenat *vistaCamping*. Desprès crida el mètode ***gestioCamping()*** de l'objecte *vistaCamping*, on es troba el bucle principal de l'aplicació.

4.3 Codi Base: Classe *Menu* i *ExempleMenu*

Al codi base disposeu de la classe ***Menu*** i un exemple de com s'utilitza la classe ***Menu*** a ***ExempleMenu***. La classe ***Menu*** gestiona un menú genèric, permetent mostrar la llista d'opcions i controlar la selecció d'una opció per part de l'usuari. Podeu executar la classe ***ExempleMenu*** per veure l'output i veure com s'utilitza la classe ***Menu***.

4.4 Codi Base: Interficies *InAcces, InAllotjament, InLListaAllotjaments, InLlistaAcessos, InLlistaTasquesManteniment*

Les interfícies ***InAcces***, ***InAllotjament, inTascaManteniment***, ***InLListaAllotjaments***, ***InLlistaAcessos*** i ***InLlistaTasquesManteniment*** donaran forma a algunes de les classes que s'han de desenvolupar durant el projecte.

5. Desenvolupament del Projecte

Les classes necessàries per a desenvolupar el sistema de gestió del càmping Green han de formar part del paquet **prog2.model**.

En tot moment tingueu en compte que es valorarà la modularitat del vostre codi, la reutilització i el seguiment de les bones pràctiques de programació. Com a regla general, totes les classes del model hauran de tenir un constructor i un conjunt de mètodes *getters* y *setters*.

5.1 Classe *VistaCamping*

Aquesta classe és responsable de la interacció amb l'usuari.

Tal com es pot veure al mètode *main* de la classe ***IniciadorCamping*** on s'instancia l'objecte de la classe ***VistaCamping***, la classe ***VistaCamping*** ha de tenir un mètode *gestioCamping*() que gestionarà el menú principal de l'aplicació. El menú s'implementarà fent servir la classe ***Menu*** del codi base.

La classe ***VistaCamping*** haurà de contenir un atribut privat de tipus ***Camping***, anomenat *camping*, que contindrà tota la informació sobre el càmping. Al constructor de la classe ***VistaCamping*** s'haurà d'instanciar aquest atribut i cridar al mètode *inicialitzaDadesCamping* de la classe ***Camping*** (veure secció 5.2.).

5.2 Classe *Camping*

La classe ***Camping*** conté tota la informació sobre el càmping de la mateixa manera que a la pràctica 1. És la classe principal del model i a través d'aquesta classe, la vista (és a dir, la classe ***VistaCamping***) podrà demanar informació sobre el càmping i canviar el seu estat.

Aquesta classe té un mètode *afegirTascaManteniment* i *completarTascaManteniment* (amb les signatures indicades a continuació). El primer mètode farà el següent: recuperar l'allotjament amb l'identificador donat, invocar el mètode per *afegirTascaManteniment* de l'objecte de tipus ***LlistaTasquesManteniment*** i per últim actualitzar l'estat de tots els accessos, mitjançant el mètode *actualitzaEstatAccessos* de la classe ***LlistaAccessos***. El segon mètode, que s'usarà per completar les tasques, farà el següent: recuperar la tasca amb el número donat, invocar el mètode per *completarTascaManteniment* de l'objecte de tipus ***LlistaTasquesManteniment*** i per últim actualitzar l'estat de tots els accessos, mitjançant el mètode *actualitzaEstatAccessos* de la classe ***LlistaAccessos***.

```
public void afegirTascaManteniment(int numTasca, String tipus, String idAllotjament, String data, int dies) throws ExcepcioCamping;

public void completarTascaManteniment(int numTasca) throws ExcepcioCamping;
```

La resta de la implementació l'hauràs de fer de forma autònoma, seguint l'enunciat, el codi proporcionat i el teu criteri de programador/a.

Inicialització de les dades del Càmping *Green*: En el codi base del Campus Virtual pots trobar un mètode *inicialitzaDadesCamping* (dins del fitxer *inicialitzaDadesCamping.txt*) que inicialitza les dades del càmping amb la informació de les Taules 1 i 2. Cal copiar aquest mètode en la classe ***Camping***. Aquesta funció s'haurà de cridar des del constructor de la vista (***VistaCamping***).

5.3 Classe *Acces, AccesAsfalt, AccesTerra i classes* filles

Per modelar els diferents tipus d'accessos utilitzarem el mecanisme d'herència. S'haurà de crear una classe abstracta anomenada ***Acces***, de la següent manera:

```
public abstract class Acces implements InAcces{

}
```

Aquesta classe contindrà els atributs comuns a tots els accessos, és a dir, nom, accessibilitat, estat (un boolean per indicar obert o tancat), i la llista dels allotjaments als que permet l'accés.

També crearem les classes abstractes ***AccesAsfalt*** i ***AccesTerra*** (subclasse de ***Acces***) per guardar la seva informació dels metres quadrats d'asfalt i longitud, respectivament.

A continuació s'hauran de crear 2 classes filles d'***AccessAsfalt*** anomenades ***CamiAsfalt*** i ***CarreteraAsfalt*** i 2 classes filles d'***AccesTerra*** anomenades ***CamiTerra*** i ***CarreteraTerra***. Aquestes 4 classes implementaran cadascuna el seu mètode isAccessibilitat(), que retorna si l'accés proporciona accessibilitat o no tal com està explicat en la secció 1. Els mètodes *getters* i *setters* així com el mètode *toString()* s'hauran de definir en els accessos que afegeixen atributs (carretera asfaltada i carretera de terra). Aquest *toString* haurà de cridar al mètode *toString* de la superclasse.

5.4 Classe *Allotjament i classes filles*

La classe ***Allotjament*** i les seves classes filles s'estructuren de la mateixa forma que en la pràctica 1, amb la diferència que incorporem dos atributs: un per controlar si l'allotjament està operatiu (estat de l'allotjament) i un altre que indiqui la il·luminació d'aquest (estat de la il·luminació). Per determinar si un allotjament està operatiu, es seguirà el criteri especificat a la secció 1 (no operatiu en cas d'haver-hi alguna tasca de manteniment activa), i no els criteris especificats a l'anterior pràctica. A part dels *getters* i *setters* i el *toString,* els mètodes de la classe ***Allotjament*** estan definits en la interfície ***InAllotjament***.

5.5 Classe *TascaManteniment*

La classe ***TascaManteniment*** conté la informació d'una tasca de manteniment: el número de la tasca, l'allotjament, la data (que pot ser un String), el tipus de tasca (que es pot definir fent servir el ***enum TipusTascaManteniment*** que s'ha de definir en la pròpia classe), i el nombre de dies esperat per completar-la. Els mètodes de la classe ***TascaManteniment*** estan definits en la interfície ***InTascaManteniment***.

```
public static enum TipusTascaManteniment {
      Reparacio,
      Neteja,
      RevisioTecnica,
      Desinfeccio
   };
```

A més del mètode constructor, el mètode *toString* i els mètodes *getters* i *setters* dels atributs, s'haurà de definir el següent mètode que retorna la informació del percentatge de la il·luminació a la que s'ha de deixar l'allotjament on s'ha afegit la tasca segons el tipus de la mateixa tasca:

```
public String getIlluminacioAllotjament();

}
```

5.6 Classes *LlistaAllotjaments, LlistaAccessos, LlistaTasquesManteniment.*

S'ha de crear la classe ***LlistaAllotjaments, LListaAccessos i LlistaTasquesManteniment*** que, com el seu nom indica, té com a objectiu guardar i gestionar una llista d'allotjaments, accessos i tasques de manteniment, respectivament. Cal que aquestes classes tinguin un atribut privat de tipus *ArrayList<Allotjament>*, *ArrayList<Acces>* i *ArrayList<TascaManteniment>*, respectivament, que s'inicialitzarà en el constructor de la classe. A més, la forma d'aquestes classes ve donada en les interfícies ***InLListaAllotjaments***, ***InLlistaAcessos*** i ***InLlistaTasquesManteniment***.

Tingueu en compte que un allotjament no pot tenir més d'una tasca de manteniment i això s'ha de controlar en el moment que es vol afegir una nova tasca (opció 7 del menú).

Llegiu la descripció dels mètodes que hi ha als comentaris de les interfícies per seguir les indicacions i fer la implementació de les classes. Per exemple, el mètode ***afegirTascaManteniment*** de ***LListaTasquesManteniment*** haurà de comprovar i avisar si l'allotjament ja té una tasca activa o si el *tipus* de tasca que es vol afegir no existeix.

5.7 Persistència de dades

Amb les **opcions 11 i 12 del menú** volem poder guardar les dades (***Camping***) a un fitxer en disc i poder-les carregar posteriorment.

Per fer-ho implementarem els següents mètodes guardar i carregar a la classe ***Camping:***

```
public void guardar(String camiDesti) throws ExcepcioCamping;

public static Camping carregar(String camiOrigen) throws ExcepcioCamping;
```

I seguirem els següents passos:

1. Obtenir la ruta al fitxer on voleu guardar les dades o des del qual voleu carregar-les. Necessitareu guardar aquesta informació en un objecte de tipus File. Per exemple, si volem utilitzar el fitxer "Camping.dat" (Nota: no cal que existeixi a disc, es crea en el moment de fer new), farem:

```
File fitxer = new File("Camping.dat");
```

2. L'accés de lectura i escriptura a un fitxer es fa mitjançant *Streams*. Per llegir d'un fitxer utilitzarem un objecte de tipus ***FileInputStream***, i per escriure a un fitxer utilitzarem un objecte de tipus ***FileOutputStream***:

```
FileInputStream fin=new FileInputStream(fitxer);

FileOutputStream fout= new FileOutputStream(fitxer);
```

3. Finalment, existeixen objectes per gestionar la lectura i escriptura d'un objecte a un *Stream*. Per escriure un objecte utilitzarem un objecte de tipus ***ObjectOutputStream***, mentre que per llegir-lo utilitzarem un objecte de tipus ***ObjectInputStream***:

```
ObjectOutputStream oos = new ObjectOutputStream(fout);

ObjectInputStream ois = new ObjectInputStream(fin);
```

No oblideu tancar correctament els objectes d'accés als fitxers.

Teniu més informació sobre *Streams* en el document d'ajuda a la pràctica 2 al Campus Virtual.

5.8 Excepcions

Tal com heu fet a la pràctica 1, heu d'utilitzar excepcions per fer la gestió dels errors, com per exemple, l'error d'afegir una tasca sobre un allotjament que ja té una tasca de manteniment actualment o l'error de llistar els allotjaments quan no hi ha cap allotjament a la llista.

Creeu la classe ***ExcepcioCamping*** dins el [paquet Vista] (***prog2.vista***) per representar aquests errors particulars. A la part del codi on es produeix un problema a controlar, heu de crear un objecte d'aquesta excepció i llançar-la, fent servir ***throw*** dins del mètode corresponent. La paraula ***throws*** al final de la capçalera del mètode ens permetrà delegar la captura d'aquesta excepció fins al mètode on es vol gestionar. Utilitzeu el mateix constructor de l'excepció per introduir-hi el missatge d'error adequat i específic de cada error. Finalment, a la vista, feu servir *try-catch* per capturar la excepció i llavors informar a l'usuari de l'error produït.

Fixeu-vos que la majoria dels mètodes definits a les interfícies permeten llançar ***ExcepcioCamping***, però a l'hora de fer la sobreescriptura d'aquests mètodes a les vostres classes no heu d'afegir el "throws ExcepcioCamping;" a la signatura dels vostres mètodes si no ho necessiteu, és a dir, si no heu de llançar una excepció. El fet de posar-ho a la definició dels mètodes de les interfícies és per fer-los més genèrics.

5.9 Classes de test

Per comprovar el correcte funcionament del codi implementat és necessari córrer un conjunt de casos de prova (Unit Tests) que heu de definir en diverses classes de test corresponents a cadascuna de les classes del model i que hauran d'estar creades a la carpeta *test*. Aquests tests han de complementar els tests bàsics que us proporcionem per a les classes ***Acces***, ***Allotjament*** i ***TascaManteniment***, els quals us ajudaran a definir el constructor de cadascuna d'aquestes classes

Un cas de prova (Unit Test Case) és una part de codi, que assegura que una altra part del codi (un mètode) funciona com s'esperava. Un cas de prova es caracteritza per una entrada coneguda i un resultat esperat.

JUnit és un marc de proves unitàries per al llenguatge de programació Java (https://junit.org/junit5/, https://www.jetbrains.com/help/idea/junit.html).

Teniu un document de suport sobre JUnit al Campus Virtual.

6. Format del Lliurament

El lliurament consistirà en tot el codi generat en els diferents punts de l'enunciat, juntament amb la documentació especificada en aquest apartat.

En concret, cal generar un fitxer comprimit (ZIP) amb el nom dels dos membres de la parella: **Cognom1Nom1Cognom2Nom2_Pr2**, que contingui:

A. El projecte sencer de IntelliJ

Tot el codi generat ha d'estar correctament comentat per a poder executar el **JavaDoc** (consultar el document de suport en el Campus Virtual) i generar automàticament la documentació en línia del codi.

B. La memòria de la pràctica (amb un màxim de 6 pàgines).

La memòria ha de contenir una portada, els autors i els punts següents:

1. Explicar **breument** les classes implementades.
2. Dibuixar el diagrama de relacions entre les classes que has utilitzat a la pràctica. No cal incloure la llista d'atributs i mètodes.
3. Explicar quins són els atributs de la classe ***Camping*** i perquè.
4. Explicar perquè el mètode isAccessibilitat de la classe ***Acces*** és abstracte.

5. Creus que l'opció 8 del menú per completar una tasca de manteniment es podria implementar demanant el nom de l'allotjament de la tasca, en lloc del número de la tasca que es vol completar?

6. Per què creus que es demana que el mètode carregar de la classe ***Camping*** sigui *static*?

7. Detallar les proves realitzades per comprovar el correcte funcionament de la pràctica, resultats obtinguts i accions derivades.

8. Observacions generals.

7. Data Límit del Lliurament

Consultar la tasca oberta pel lliurament al Campus Virtual.